



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Московский государственный технический университет имени Н.Э.
Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Фундаментальные науки
КАФЕДРА ФН12

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент Иртюга Илья Владимирович
Группа ФН12-81Б
Тип практики преддипломная
Название предприятия МГТУ им. Н.Э. Баумана

Студент	_____	Иртюга И.В.
	подпись, дата	фамилия, и.о.
Руководитель практики	_____	Велищанский М.А.
	подпись, дата	фамилия, и.о.
Оценка	_____	

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Анализ предметной области и постановка задачи	5
2 Обзор методов решения задачи 3D Bin Packing	9
3 Разработанное решение Smart 3D Packing	11
4 Эксперименты, метрики и анализ результатов	17
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	25

ВВЕДЕНИЕ

Задача упаковки объектов в ограниченный контейнер является одной из классических задач комбинаторной оптимизации. В трехмерном варианте требуется разместить набор прямоугольных параллелепипедов внутри контейнера или на паллете так, чтобы объекты не пересекались и не выходили за заданные границы. В прикладной логистике такая задача возникает при сборке заказов, паллетизации товаров, загрузке транспорта и планировании складских операций.

В крупном ритейле качество паллетизации напрямую влияет на стоимость перевозки и устойчивость товарного потока. Недостаточно плотная укладка приводит к перевозке незаполненного объема, а неучет веса, хрупкости и устойчивости увеличивает риск повреждения товаров. Поэтому практическая постановка 3D Bin Packing существенно шире чисто геометрической задачи: необходимо одновременно учитывать размеры, массу, допустимые ориентации, опору снизу, возможность штабелирования и время расчета.

Тема производственной практики связана с проектом Smart 3D Packing. Условие проекта формулирует задачу построения алгоритмического модуля 3D Pallet Packing Optimizer, который по описанию паллеты и списка коробов возвращает план размещения с координатами и ориентациями каждого короба. Согласно условию, решение проходит двухэтапную проверку: сначала контролируются жесткие ограничения физической корректности, затем вычисляется взвешенная метрика качества.

Актуальность работы обусловлена тем, что задача 3D Bin Packing относится к NP-трудным задачам [1], а практически значимые экземпляры содержат десятки и сотни объектов. Полный перебор быстро становится неприменимым, поэтому в реальных системах используются эвристики, локальный поиск, метаэвристики и гибридные методы [2]. В исследуемом проекте реализованы несколько таких подходов, что позволяет сравнить их в едином экспериментальном контуре.

Цель практики – исследовать эвристические и ML-подходы к решению задачи 3D Bin Packing в проекте Smart 3D Packing и определить, какие методы обеспечивают лучший баланс качества укладки и времени расчета.

Для достижения цели решаются следующие задачи:

1. изучить условие задачи Smart 3D Packing, требования к входным и выходным

- данным и правила валидации;
2. формализовать ограничения и метрику качества упаковки;
 3. проанализировать структуру репозитория и восстановить pipeline решения;
 4. описать генерацию синтетических датасетов и состав используемых наборов данных;
 5. рассмотреть реализованные алгоритмы: greedy, MaxRects, LNS, GAN+GA, brute force и многопаллетное расширение;
 6. провести сравнительный анализ сохраненных экспериментальных результатов;
 7. сформулировать ограничения текущей реализации и направления дальнейшего развития.

Объект исследования – алгоритмическая система трехмерной укладки коробов на паллету.

Предмет исследования – эвристические, метаэвристические и ML-ориентированные методы построения допустимой и качественной укладки в задаче 3D Bin Packing.

Практическая значимость выполненной работы заключается в том, что исследуемая система объединяет генератор данных, набор решателей, валидатор, средства визуализации и сохраненные benchmark-результаты. Такая структура может быть использована как основа для дальнейших исследований алгоритмов паллетизации и для прототипирования прикладного логистического модуля.

Отчет состоит из введения, четырех разделов основной части, заключения и списка использованных источников. В первом разделе рассматриваются предметная область и постановка задачи. Во втором разделе приведен обзор методов решения 3D Bin Packing. В третьем разделе описана архитектура проекта Smart 3D Packing и реализация решателей. В четвертом разделе представлены датасеты, эксперименты, метрики и интерпретация результатов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Паллетизация в складской логистике состоит в формировании устойчивой грузовой единицы из множества товарных коробов. На практике решение должно быть не только плотным, но и выполнимым для транспортировки: тяжелые товары не должны разрушать хрупкие, нестабильные слои не должны приводить к падению груза, а ограничения по высоте и массе должны соблюдаться для совместимости со складским оборудованием и транспортом.

В условии Smart 3D Packing отдельно выделены типичные проблемы ручной или простой эвристической сборки: низкая утилизация объема, риск повреждений из-за неправильного распределения веса и нестабильность конструкции. Эти факторы показывают, что задача имеет многокритериальный характер. Один только максимум занятого объема не гарантирует хорошего решения, если оно нарушает устойчивость, портит товар или требует слишком долгого расчета.

С математической точки зрения задача относится к семейству bin packing и cutting/packing problems [4]. В классической трехмерной постановке требуется разместить прямоугольные объекты в одном или нескольких контейнерах. Даже более простые варианты bin packing являются вычислительно сложными, а добавление трехмерной геометрии и физических ограничений резко увеличивает размер пространства решений [1, 4].

1.2 Входные данные

Вход одной задачи представлен JSON-объектом. Он содержит описание паллеты и список типов коробов. Для паллеты задаются:

- длина `length_mm`;
- ширина `width_mm`;
- максимальная высота укладки `max_height_mm`;
- максимальная масса `max_weight_kg`.

Для каждого типа короба задаются идентификатор SKU, описание, габариты, масса, количество экземпляров и дополнительные признаки:

- `strict_upright` – запрет поворота вокруг горизонтальных осей;
- `fragile` – признак хрупкости;
- `stackable` – возможность ставить другие короба поверх данного.

В генераторе проекта используются типовые архетипы товаров фудритейла: вода, сахар, вино, бананы, чипсы, яйца и консервы. Размеры и массы этих архетипов варьируются небольшим случайным шумом, что позволяет получать воспроизводимые, но не полностью одинаковые задачи.

1.3 Выходные данные

Результатом работы решателя является JSON-ответ, содержащий идентификатор задачи, версию решателя, время расчета и список размещенных коробов. Для каждого размещенного экземпляра указываются:

- `sku_id` и `instance_index`;
- координаты нижнего ближнего левого угла `x_mm`, `y_mm`, `z_mm`;
- фактические размеры после поворота;
- код ориентации `rotation_code`.

Если часть объектов не размещена, она фиксируется в массиве `unplaced` с указанием количества и причины: превышение массы, высоты или отсутствие места. Такая структура удобна для последующей валидации и анализа: ответ содержит как геометрию решения, так и информацию о неполноте размещения заказа.

1.4 Жесткие ограничения

В проекте реализован валидатор `validator.py`, который проверяет физическую корректность решения. Нарушение любого жесткого ограничения делает решение невалидным для конкретной задачи.

Основные ограничения следующие:

1. каждый размещенный короб должен целиком находиться внутри габаритов паллеты;
2. объемы любых двух коробов не должны пересекаться;
3. суммарная масса размещенных товаров не должна превышать грузоподъемность паллеты;

4. если `strict_upright=true`, фактическая высота короба должна совпадать с исходной высотой;
5. короб, расположенный выше пола паллеты, должен иметь опору снизу не менее чем на 60% площади основания согласно формуле (1);
6. фактические размеры размещенного короба должны быть перестановкой исходных размеров, что исключает некорректное изменение габаритов.

Условие опоры записывается следующим образом:

$$\frac{S_{\text{support}}}{S_{\text{base}}} \geq 0.6, \quad (1)$$

где S_{base} – площадь основания проверяемого короба, а S_{support} – суммарная площадь пересечения его основания с верхними гранями коробов, лежащих непосредственно ниже.

1.5 Метрика качества

Если решение прошло жесткие проверки, валидатор вычисляет итоговую метрику:

$$\text{score} = 0.50U_{\text{vol}} + 0.30C_{\text{items}} + 0.10F_{\text{frag}} + 0.10T_{\text{time}}. \quad (2)$$

Компонента утилизации объема равна

$$U_{\text{vol}} = \frac{\sum_{i \in P} l_i w_i h_i}{LWH}, \quad (3)$$

где P – множество размещенных коробов, l_i, w_i, h_i – их фактические размеры, L, W, H – габариты паллеты.

Полнота заказа определяется как

$$C_{\text{items}} = \frac{N_{\text{placed}}}{N_{\text{total}}}, \quad (4)$$

где N_{placed} – число размещенных экземпляров, N_{total} – общее число экземпляров во входной задаче.

Оценка хрупкости задается формулой

$$F_{\text{frag}} = \max(0, 1 - 0.05n_{\text{viol}}), \quad (5)$$

где n_{viol} – число случаев, когда короб массой более 2 кг расположен непосредственно над хрупким коробом с ненулевым перекрытием в плоскости XU .

Временной балл имеет ступенчатый вид: при времени до 1 секунды он равен 1.0, при времени от 1 до 5 секунд – 0.7, при времени от 5 до 30 секунд – 0.3, при большем времени – 0.0. Такая метрика поощряет алгоритмы, которые дают хороший результат в режиме, близком к реальному времени.

1.6 Критерии качества упаковки

Формула (2) показывает, что 80% итогового score формируют плотность упаковки и доля размещенных товаров. Однако оставшиеся 20% также существенны, поскольку они отражают практическую применимость решения. Метод, который достигает высокой плотности за счет нарушения хрупкости или слишком большого времени расчета, может уступить более сбалансированному подходу.

Таким образом, в работе используются следующие критерии сравнения:

- средний итоговый score на наборе задач;
- средняя утилизация объема;
- средняя полнота размещения;
- средний балл хрупкости;
- средний временной балл и фактическое время решения;
- число валидных решений.

1.7 Выводы по разделу

Задача Smart 3D Packing является прикладной модификацией 3D Bin Packing, в которой геометрическая упаковка дополняется логистическими ограничениями. Наиболее важными особенностями постановки являются поддержка поворотов, проверка опоры, учет массы, запрет некорректного размещения тяжелых товаров над хрупкими и ограничение времени. Это делает задачу удобной для сравнения как быстрых эвристик, так и более сложных метаэвристических и ML-подходов.

2 ОБЗОР МЕТОДОВ РЕШЕНИЯ ЗАДАЧИ 3D BIN PACKING

Задача 3D Bin Packing относится к классу NP-трудных задач упаковки, поэтому в практических системах редко используется полный перебор всех размещений. В ходе практики рассматривались методы, которые дают допустимое решение за ограниченное время и могут быть проверены единым валидатором: последовательные greedy-эвристики, послойные схемы MaxRects, метаэвристика Large Neighborhood Search, генетический подход с генеративной компонентой и точный поиск для малых экземпляров [2–4].

Классические greedy-алгоритмы строят укладку последовательно: объекты сортируются по выбранному приоритету, после чего для каждого перебираются допустимые ориентации и кандидатные позиции. В проекте такая логика реализована через extreme points и gravity drop. Метод быстр и удобен как baseline, но плохо исправляет ранние неудачные решения: если крупный или хрупкий товар размещен неудачно, последующие объекты уже подстраиваются под эту структуру.

Послойный MaxRects-подход использует идею двумерной упаковки прямоугольников внутри слоя. В проекте он адаптирован для паллетизации: выбирается высота слоя, основания коробов размещаются на плоскости, затем алгоритм переходит к следующему уровню. Такой решатель быстро достигает высокой утилизации объема, однако из-за послойной структуры хуже учитывает взаимное расположение хрупких и тяжелых товаров.

Large Neighborhood Search устраняет главный недостаток greedy-схем: вместо локального продолжения уже построенной укладки он периодически удаляет часть размещенных коробов и восстанавливает решение заново. В проекте LNS использует greedy-фазу для старта, несколько destroy-операторов, repair на основе beam search и принятие кандидатов с элементами simulated annealing. По сохраненным результатам именно LNS дает лучший средний итоговый score на общих наборах.

Генетические и ML-ориентированные методы рассматриваются как способ расширить пространство поиска. Хромосома задает порядок объектов и выбор ориентаций, а декодер строит конкретную укладку. Реализованный вариант GAN+GA не использует тяжелые фреймворки обучения: генеративная часть

написана на NumPy и влияет на разнообразие популяции и fitness. Такой подход интересен как направление развития, но в текущих экспериментах уступает LNS по среднему score [5].

Точный поиск и branch-and-bound используются только для малых задач из каталога `brute_force_vs_algo`. Они полезны как исследовательская точка сравнения: позволяют проверить, насколько эвристические решения близки к более полному перебору. Для рабочих размеров такой подход становится слишком дорогим по времени, поэтому основной практический акцент сделан на LNS и быстрых базовых решателях.

3 РАЗРАБОТАННОЕ РЕШЕНИЕ SMART 3D PACKING

3.1 Структура репозитория

Репозиторий проекта организован как экспериментальная среда для исследования алгоритмов паллетизации [6]. Основные каталоги и файлы приведены в таблице 1.

Таблица 1 – Основные компоненты проекта

Компонент	Назначение
datasets	готовые single-pallet наборы задач в формате JSON
dataset_generation	генератор синтетических сценариев фуд-ритейла
solvers	основные single-pallet решатели: greedy, MaxRects, LNS, GAN+GA
validator.py	проверка жестких ограничений и расчет итоговой метрики
results	сохраненные результаты запусков решателей и рассчитанные метрики
brute_force_vs_algo	сравнение эвристик с branch-and-bound на малых задачах
multipallet	расширение на случай нескольких паллет
vizualizator	интерактивная визуализация укладок
benchmark_report.py, plot_benchmark.py	агрегация результатов и построение графиков

Общий pipeline проекта показан на рис. 1. Сначала формируется или загружается входной JSON, затем один из решателей строит укладку, валидатор проверяет корректность и вычисляет метрики, после чего результаты сохраняются и могут быть визуализированы или агрегированы в benchmark-отчет.

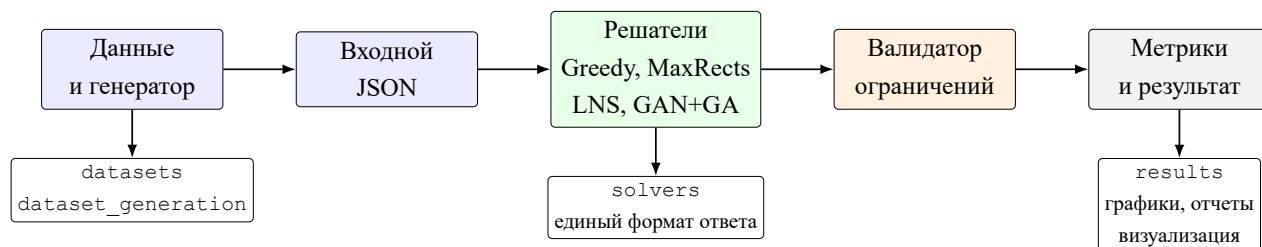


Рисунок 1 – Архитектура экспериментального контура проекта

3.2 Генерация данных

Генератор `dataset_generation/generator.py` использует набор товарных архетипов. Для каждого архетипа заданы базовые размеры, масса,

признак вертикальной ориентации и хрупкость. При создании конкретной задачи размеры и масса варьируются небольшим случайным шумом, а количество экземпляров выбирается из диапазона.

В проекте реализованы сценарии:

- `heavy_water` – большое число тяжелых упаковок воды и сахара;
- `fragile_tower` – комбинация бананов, чипсов и яиц с хрупкими товарами;
- `liquid_tetris` – вода, вино и консервы с ограничениями ориентации;
- `high_count_mixed` – много разнотипных товаров;
- `heavy_fragile_mix` – тяжелые и хрупкие товары в одном заказе;
- `random_mixed` – случайная смесь нескольких архетипов.

Состав датасетов приведен в таблице 2. Эти данные были получены автоматическим анализом файлов из каталога `datasets`.

Таблица 2 – Характеристики используемых датасетов

Датасет	Задач	Ср. объектов	Мин.	Макс.	Ср. SKU
<code>dataset_100.json</code>	100	94.6	35	194	3.4
<code>dataset_150.json</code>	150	94.9	36	185	3.3
<code>dataset_200_2.json</code>	200	94.7	33	194	3.4
<code>dataset_benchmark.json</code>	200	140.0	31	316	3.7

Распределение задач по сценариям для каждого датасета показано на рис. 2.

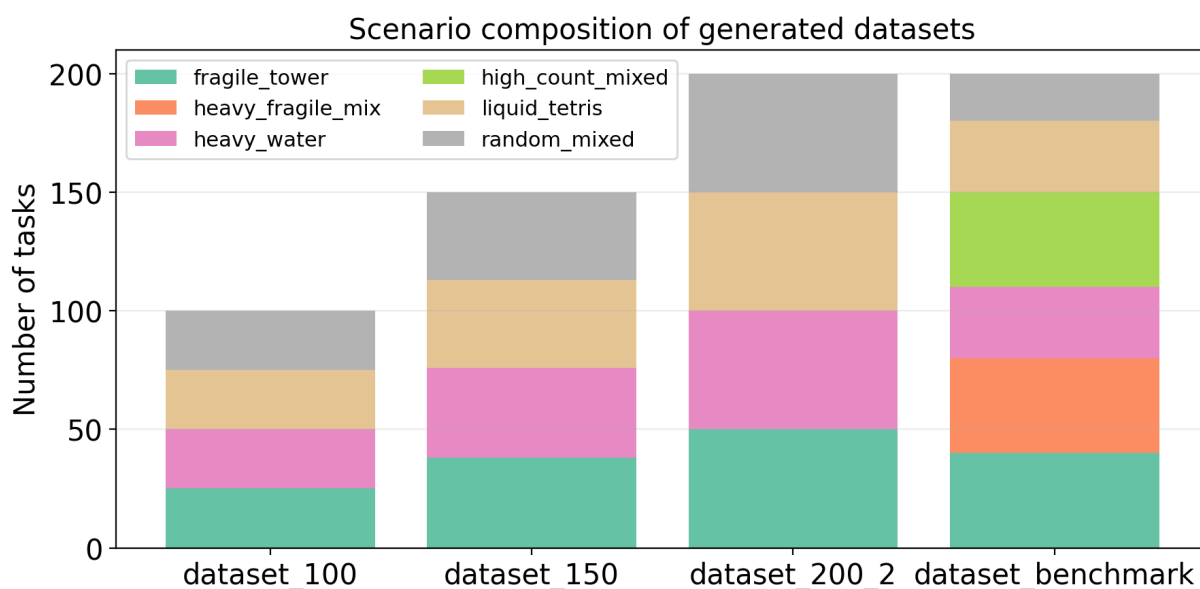


Рисунок 2 – Сценарный состав сгенерированных наборов задач

3.3 Валидатор и единая оценка

Файл `validator.py` является центральным элементом экспериментальной методологии. Он не зависит от конкретного решателя и выполняет одинаковые проверки для всех ответов. Сначала валидатор восстанавливает геометрию размещенных коробов по координатам и размерам, затем проверяет `anti-cheat` условие: фактические размеры должны быть перестановкой исходных.

После этого последовательно проверяются масса, границы паллеты, ограничение `strict_upright`, трехмерные пересечения и опора. Только валидные решения получают метрики. Такой подход исключает ситуацию, когда один метод оценивается по более мягким правилам, чем другой.

3.4 Greedy-решатель

Файл `solvers/greedy_algorithm.py` реализует последовательный алгоритм на `extreme points`. Количество каждого SKU разворачивается в список отдельных экземпляров. Затем объекты сортируются по объему, причем хрупкие и нештабелируемые товары получают специальный приоритет, чтобы уменьшить риск плохой структуры укладки.

Для каждого объекта перебираются допустимые повороты. Функция `get_rotations` возвращает ориентации из множества `LWH, LHW, WLH, WHL, HLW, HWL`; для `strict_upright` остаются только повороты с исходной высотой. Проверки пересечений, опоры и хрупкости ускорены пространственной сеткой `PlacedGrid`, которая ограничивает число уже размещенных коробов, участвующих в геометрических проверках.

Преимущество `greedy` – скорость и простота. На всех сохраненных `single-pallet` наборах средний временной балл равен 1.0. Недостаток – невозможность перестроить ранние решения, что особенно заметно на `dataset_benchmark.json`: средний `item coverage` снижается до 0.4906.

3.5 MaxRects-решатель

Файл `solvers/solve_conditions.py` реализует послойный подход. Для каждого слоя выбирается высота-кандидат, после чего основания коробов упаковываются двумерным `MaxRects`-подобным алгоритмом. Метод сор-

тирует товары по массе и объему, пытается заполнить слой, затем переходит к следующей высоте.

Этот решатель очень быстр: среднее время на контрольном наборе составляет около 52 мс. Он достигает высокой утилизации объема, но заметно проигрывает по хрупкости. Такое поведение объясняется тем, что послойная плотность и сортировка по весу не всегда согласуются с правилом размещения хрупких товаров верхним слоем.

3.6 LNS-решатель

Файл `solvers/lms_solver.py` содержит наиболее сильный решатель проекта. Его работа строится как цикл: построение стартового решения, удаление части размещенных коробов, стабилизация опоры, восстановление решения `beam search`, оценка кандидата, принятие по итоговому `score` и `simulated annealing`, затем проверка бюджета времени.

Сначала LNS строит начальное решение. В отличие от одиночного `greedy`-запуска, используется несколько детерминированных порядков и случайные рестарты в пределах бюджета `greedy_phase_s`. Затем запускается основной цикл `destroy/repair`. В реализации присутствуют `destroy`-эвристики:

- случайное удаление размещенных коробов;
- удаление верхних коробов;
- удаление пространственного кластера;
- удаление коробов, связанных с нарушениями хрупкости;
- удаление коробов, близких по объему к непо помещенным;
- полный рестарт;
- удаление малых размещенных коробов.

После разрушения вызывается стабилизация частичного решения: если некоторые короба потеряли достаточную опору, они также удаляются. Этап `repair` основан на `beam search`. Для частичных состояний применяется оценка

$$\text{partial_score} = 0.50U_{\text{vol}} + 0.30C_{\text{items}} - 0.005n_{\text{frag_viol}}, \quad (6)$$

которая ориентирует восстановление на объем, покрытие и умеренный штраф за потенциальные нарушения хрупкости.

Ключевые параметры LNS приведены в таблице 3.

Таблица 3 – Основные параметры LNS-конфигурации

Параметр	Значение	Назначение
time_budget_s	4.8	общий бюджет времени
greedy_phase_s	1.5	стартовая greedy-фаза
destroy_k_fraction	0.15	доля удаляемых коробов
destroy_k_min/max	3 / 15	границы размера разрушения
beam_width	8	ширина beam search
repair_queue_limit	50	максимальная очередь восстановления
repair_restarts	2	дополнительные repair-рестарты
early_stop_patience	25	остановка при отсутствии улучшений
ils_kicks	5	число ILS-перезапусков
sa_initial_temp	0.015	начальная температура simulated annealing
sa_cooling_rate	0.92	скорость охлаждения

3.7 GAN+GA-решатель

Файл `solvers/gan_ga_solver.py` реализует гибрид генетического алгоритма и простой генеративно-сопоставительной модели. Хромосома состоит из перестановки объектов и списка выборов ориентаций. Декодер строит укладку greedy-методом с ограниченным числом extreme points.

Генетическая часть использует турнирный отбор, order crossover, скрещивание ориентаций и мутации перестановки. GAN-компонента реализована на NumPy: генератор создает приоритеты объектов, преобразуемые в перестановку, а дискриминатор оценивает похожесть решения на элитные особи. Эта оценка добавляется к fitness с малым коэффициентом.

Метод интересен как ML-ориентированная альтернатива, но в текущих экспериментах уступает LNS по среднему score на общих наборах. При этом GAN+GA сохраняет высокий fragility score и демонстрирует конкурентную утилизацию объема.

3.8 Визуализация

Каталог `vizualizator` содержит средства просмотра результатов. Визуализатор читает входной JSON и JSON с размещениями, строит трехмерное представление паллеты и позволяет анализировать слои. Этот модуль используется как вспомогательный инструмент: он помогает проверять геометрию решений после запуска валидатора и находить случаи, где формально допустимая

укладка выглядит неудачной для практического применения.

3.9 Выводы по разделу

Проект представляет собой не отдельный алгоритм, а полноценный исследовательский стенд. Его сильная сторона – единый формат данных, общий валидатор и сохраненные результаты для нескольких решателей. На уровне реализации наиболее развитым методом является LNS: он использует greedy как стартовую эвристику, но дополняет ее разрушением и восстановлением фрагментов решения, адаптивными эвристиками и simulated annealing.

4 ЭКСПЕРИМЕНТЫ, МЕТРИКИ И АНАЛИЗ РЕЗУЛЬТАТОВ

4.1 Методика эксперимента

Экспериментальная часть основана на уже сохраненных результатах из каталога `results`. Для каждого решателя и датасета в JSON-файлах содержатся ответы по отдельным задачам, рассчитанные метрики и время решения. Такой подход выбран намеренно: он исключает повторную случайную генерацию результатов и позволяет опираться на фактические данные проекта.

В основном сравнении используются четыре `single-pallet` метода:

- `greedy_algorithm`;
- `conditions` – MaxRects-подобный послойный решатель;
- `gan_ga_solver`;
- `lns_solver`.

Общими для всех четырех решателей являются три набора: `dataset_100.json`, `dataset_200_2.json` и `dataset_benchmark.json`. Для GAN+GA дополнительно есть результат на `dataset_150.json`, но он не используется в основном ранжировании, чтобы сравнение было симметричным.

4.2 Сводные результаты

Средний итоговый score на общих наборах приведен в таблице 4 и на рис. 3.

Таблица 4 – Средний итоговый score на общих наборах

Метод	<code>dataset_100</code>	<code>dataset_200_2</code>	<code>dataset_benchmark</code>
Greedy	0.6842	0.7120	0.6549
MaxRects	0.7104	0.7117	0.7124
GAN+GA	0.7012	0.7327	0.7161
LNS	0.7111	0.7434	0.7298

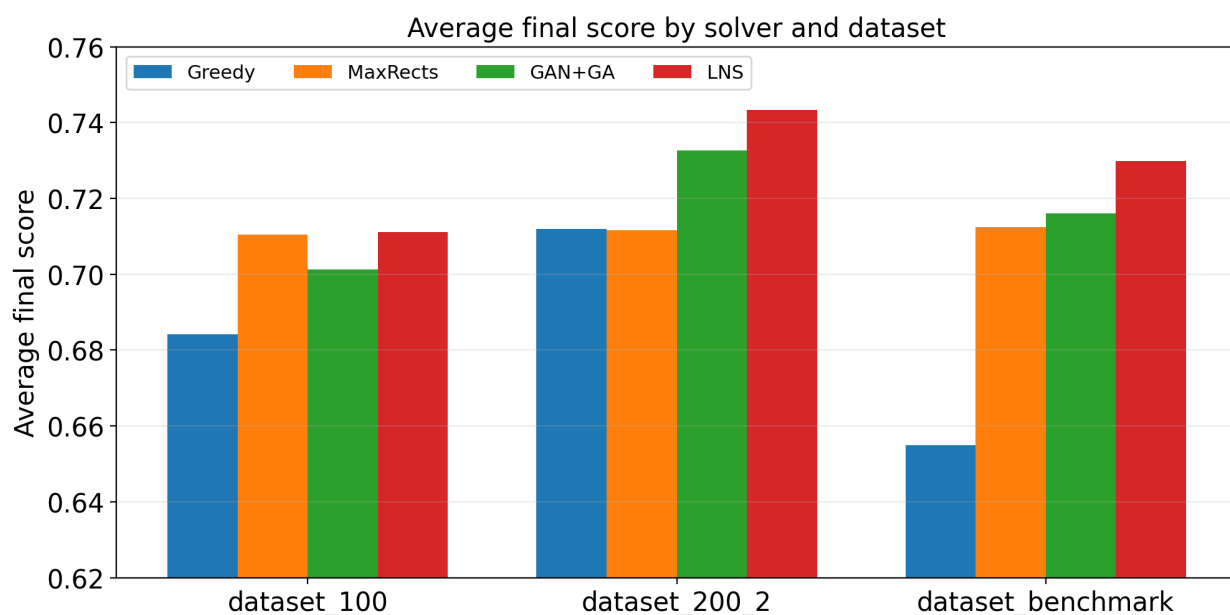


Рисунок 3 – Средний итоговый score по решателям и датасетам

LNS показывает лучший результат на всех трех общих наборах. На `dataset_100.json` его преимущество над MaxRects минимально, что объясняется меньшей сложностью задач: быстрый послойный метод уже строит достаточно сильные решения. На `dataset_200_2.json` LNS заметно обгоняет остальные методы за счет лучшей полноты размещения. На `dataset_benchmark.json`, где среднее число объектов выше и состав сценариев шире, преимущество LNS сохраняется.

4.3 Компоненты метрики

Для интерпретации результатов рассмотрим компоненты score на контрольном наборе `dataset_benchmark.json`. Данные приведены в таблице 5 и на рис. 4.

Таблица 5 – Компоненты метрики на `dataset_benchmark.json`

Метод	Score	Vol.	Coverage	Fragility	Time
Greedy	0.6549	0.6155	0.4906	1.0000	1.0000
MaxRects	0.7124	0.7513	0.6160	0.5198	1.0000
GAN+GA	0.7161	0.7156	0.5540	1.0000	0.9205
LNS	0.7298	0.7172	0.6036	0.9833	0.9175

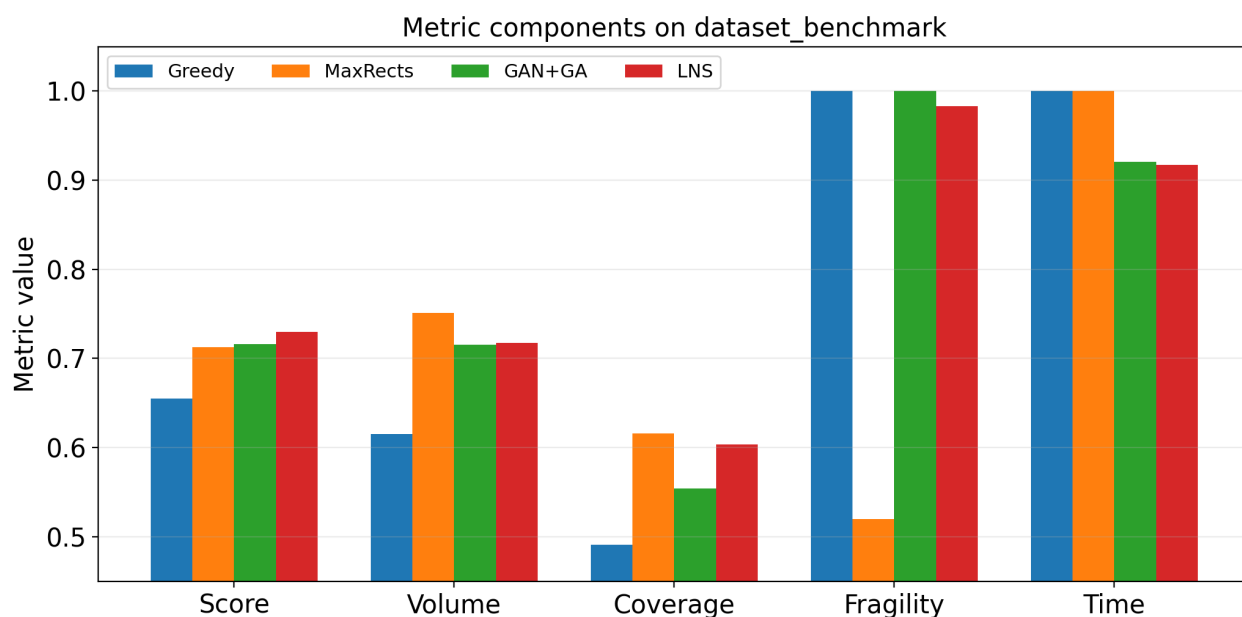


Рисунок 4 – Компоненты score на контрольном наборе

Greedy сохраняет идеальные значения по хрупкости и времени, но значительно уступает по покрытию заказа. MaxRects достигает максимальной утилизации объема и максимального покрытия на контрольном наборе, однако теряет почти половину балла за хрупкость. GAN+GA хорошо соблюдает хрупкость и заметно улучшает объем по сравнению с greedy, но уступает LNS по покрытию.

LNS не максимизирует отдельную компоненту любой ценой. Его сила заключается в балансе: утилизация объема близка к GAN+GA, покрытие близко к MaxRects, хрупкость остается высокой, а временной балл остается приемлемым. Именно этот баланс дает лучший итоговый score.

4.4 Время выполнения

Среднее фактическое время решения приведено в таблице 6. Все методы на основных наборах остаются в пределах практически приемлемого времени. Greedy и MaxRects работают быстрее всех, LNS и GAN+GA тратят около секунды на контрольном наборе.

Таблица 6 – Среднее время решения, мс

Метод	dataset_100	dataset_200_2	dataset_benchmark
Greedy	114.8	118.5	139.0
MaxRects	15.1	15.6	52.0
GAN+GA	795.3	770.4	1080.6
LNS	474.3	523.6	1062.6

Увеличение времени LNS оправдано ростом основных компонент качества. На контрольном наборе LNS теряет часть временного балла, но выигрывает по итоговой оценке. Это соответствует назначению метаэвристики: использовать дополнительное время не на случайное усложнение, а на улучшение структуры укладки.

4.5 Сравнение LNS и Greedy

НИР по проекту отдельно исследовала LNS как развитие базового greedy. Для этой пары доступны графики, построенные из сохраненных результатов. На рис. 5 показано, что LNS стабильно превосходит greedy по среднему score на трех наборах.

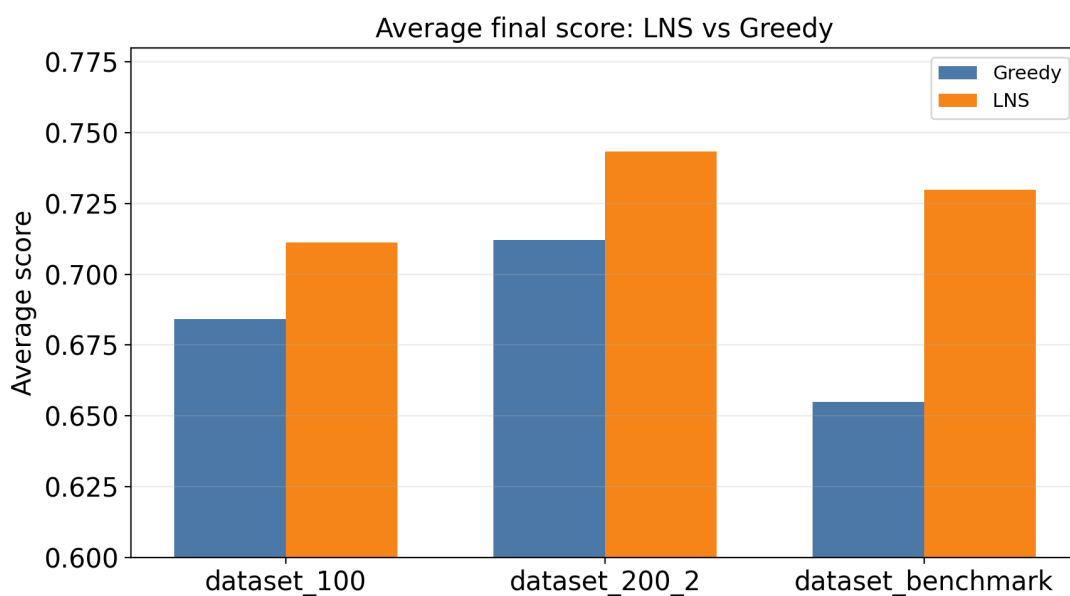


Рисунок 5 – Сравнение среднего score для LNS и greedy

Компонентное сравнение на рис. 6 показывает, что главный вклад LNS дает по утилизации объема и покрытию заказа. Небольшое ухудшение по времени и хрупкости не перекрывает этот выигрыш.

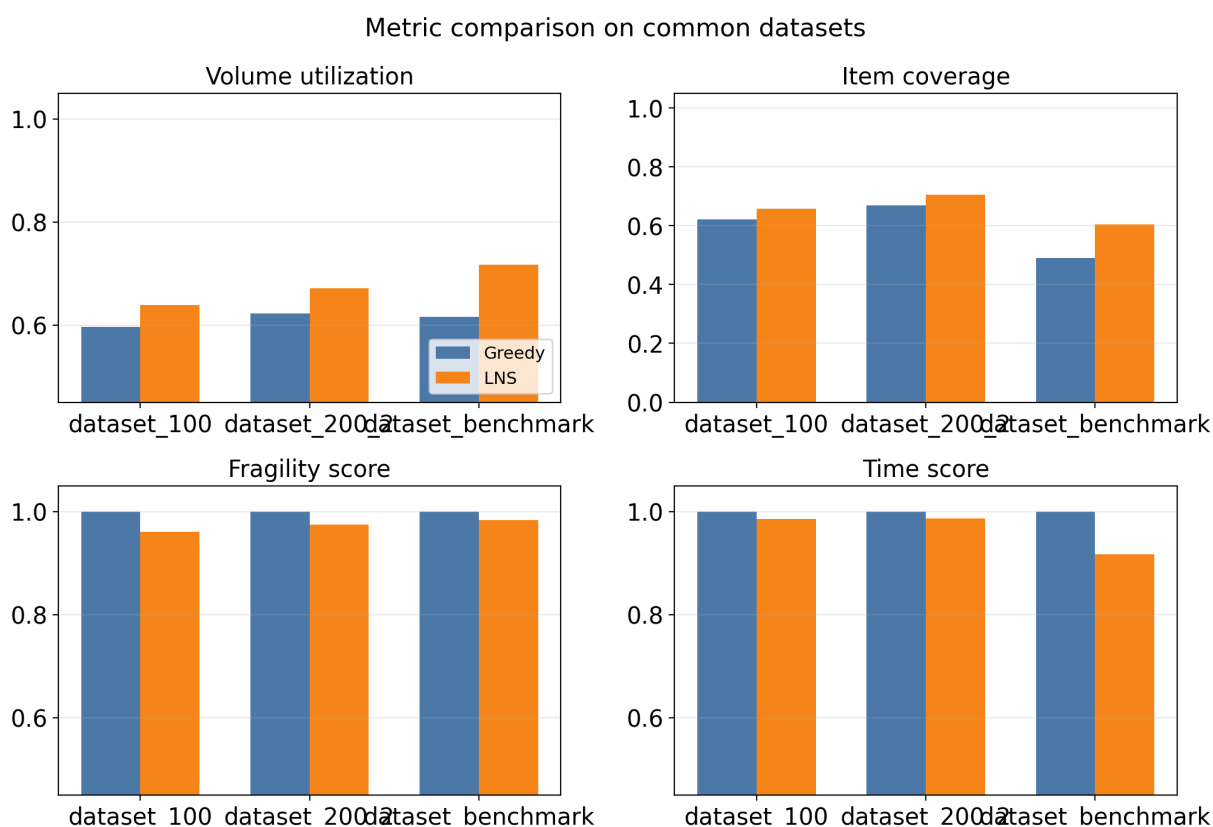


Рисунок 6 – Сравнение компонент метрики для LNS и greedy

4.6 Сравнение с brute force на малых задачах

Каталог `brute_force_vs_algo` содержит отдельные эксперименты для малых и средних наборов. Их цель – оценить поведение LNS рядом с более полным поиском. Результаты приведены в таблице 7.

Таблица 7 – Сравнение brute force и LNS на специальных наборах

Набор	Метод	Score	Coverage	Время, мс
small	Brute force	0.5563	0.9897	19.5
small	LNS	0.5602	1.0000	28.3
medium	Brute force	0.5681	0.9921	10440.7
medium	LNS	0.5780	1.0000	72.5
hard	Brute force / LDS	0.5886	0.9364	8815.0
hard	LNS	0.6501	1.0000	243.2

На этих наборах LNS не только сопоставим с перебором, но и превосходит сохраненные результаты brute force/LDS по итоговой метрике. Важно, что для `medium` и `hard` время перебора резко возрастает, тогда как LNS остается в пределах сотен миллисекунд. Это подтверждает практическую ценность мета-эвристического подхода.

4.7 Многопаллетное расширение

В каталоге `multipallet` реализовано расширение на несколько паллет. Эта постановка ближе к реальной логистике: если весь заказ нельзя разместить на одной паллете, требуется распределить оставшиеся товары на новые паллеты. В проекте есть `single-pallet` и `multi-pallet` решения для `greedy` и `LNS`.

В рамках данного отчета многопаллетный режим рассматривается как направление развития, поскольку основная метрика и основные `benchmark`-результаты сосредоточены на `single-pallet` постановке. Тем не менее наличие отдельного каталога `multipallet` показывает, что архитектура проекта допускает переход от локальной задачи плотной укладки к задаче распределения заказа между несколькими контейнерами.

4.8 Анализ ошибок и ограничений

Анализ кода и результатов позволяет выделить несколько ограничений текущей реализации.

Во-первых, модель устойчивости приближенная. Критерий 60% опоры учитывает площадь контакта, но не анализирует центр масс, распределение давления, трение и деформации упаковки. Поэтому решение может быть валидным по формальному критерию, но требовать дополнительной инженерной проверки перед промышленным применением.

Во-вторых, синтетические данные не покрывают всего разнообразия реальных заказов. В генераторе есть полезные сценарии `фуд-ритейла`, но нет, например, температурных зон, запрета соседства, приоритетов выгрузки, ограничений по слоям и требований к группировке товаров по магазинам.

В-третьих, `GAN+GA` является облегченной исследовательской реализацией. Он использует `NumPy`-модель вместо полноценного обучения на большом корпусе решений. Поэтому текущие результаты не следует трактовать как окончательный предел `ML`-подходов к задаче.

В-четвертых, `benchmark` хранит средние значения, но не содержит статистического анализа устойчивости стохастических методов по нескольким независимым запускам на каждом датасете. Для `LNS` и `GAN+GA` в дальнейшем полезно оценивать дисперсию результата.

4.9 Направления развития

Наиболее перспективные направления дальнейшей работы:

1. улучшить физическую модель устойчивости с учетом центра масс и распределения веса;
2. расширить генератор данных реальными бизнес-ограничениями;
3. добавить статистический протокол повторных запусков стохастических методов;
4. развить LNS за счет адаптивного выбора destroy/repair-операторов;
5. исследовать reinforcement learning или imitation learning для выбора порядка размещения;
6. развить многопаллетную постановку с метрикой минимизации числа паллет;
7. добавить автоматическую генерацию HTML-отчета с интерактивной 3D-визуализацией.

4.10 Выводы по разделу

Экспериментальные результаты показывают, что лучшим single-pallet методом текущего проекта является LNS. Он стабильно превосходит greedy, MaxRects и GAN+GA по среднему итоговому score на общих наборах. Причина преимуществ не в максимизации одной компоненты, а в сбалансированном улучшении объема, покрытия, хрупкости и времени. Greedy остается полезным быстрым baseline, MaxRects – сильным послойным решателем по плотности, GAN+GA – перспективным исследовательским направлением, а brute force – инструментом проверки малых экземпляров.

ЗАКЛЮЧЕНИЕ

В ходе производственной практики была исследована задача Smart 3D Packing, представляющая собой прикладную постановку 3D Bin Packing для укладки коробов на паллету. В отличие от абстрактной геометрической задачи, рассматриваемая постановка учитывает ограничения, характерные для складской логистики: габариты паллеты, грузоподъемность, допустимые повороты, опору, хрупкость, штабелируемость и время расчета.

В ходе работы были изучены условие задачи Smart 3D Packing, материалы НИР и курсовой работы, а также исходный код проекта. По результатам анализа восстановлен pipeline решения: генерация или загрузка JSON-датасета, запуск одного из решателей, валидация жестких ограничений, расчет метрик, сохранение результатов и визуализация укладки.

Были рассмотрены реализованные методы: greedy на основе extreme points и gravity drop, послойный MaxRects-решатель, Large Neighborhood Search, гибрид GAN+GA, branch-and-bound для малых задач и многопаллетное расширение. Для каждого метода описаны основная идея, роль в проекте, преимущества и ограничения.

Экспериментальная часть основана на сохраненных результатах репозитория. Сравнение на общих single-pallet наборах показало, что LNS является наиболее эффективным методом по итоговой метрике: 0.7111 на наборе из 100 задач, 0.7434 на наборе из 200 задач и 0.7298 на контрольном наборе. Его преимущество связано с тем, что метод умеет перестраивать уже найденную укладку, исправляя локально неудачные ранние решения greedy.

Greedy показал высокую скорость и идеальный временной балл, но уступил по утилизации объема и покрытию заказа. MaxRects достиг высокой плотности, но потерял качество по хрупкости. GAN+GA подтвердил перспективность гибридного ML-ориентированного поиска, однако в текущей реализации не превзошел LNS. Brute force оказался полезен для малых задач, но плохо масштабируется на более крупные экземпляры.

Таким образом, цель работы достигнута: выполнен анализ предметной области, формализована постановка, изучена реализация проекта и проведено сравнение методов. Наиболее рациональным методом для текущей single-pallet постановки является LNS; дальнейшее развитие связано с усилением destroy/repair-операторов и улучшением модели устойчивости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Martello S., Pisinger D., Vigo D. The Three-Dimensional Bin Packing Problem // Operations Research. 2000. Vol. 48, No. 2. P. 256–267. DOI: <https://doi.org/10.1287/opre.48.2.256.12386>.
2. Pisinger D., Ropke S. Large Neighborhood Search // Handbook of Metaheuristics. 2nd ed. Springer, 2010. P. 399–420.
3. Coffman E. G., Garey M. R., Johnson D. S. Approximation Algorithms for Bin Packing: A Survey // Approximation Algorithms for NP-Hard Problems. Boston: PWS Publishing, 1997. P. 46–93.
4. Lodi A., Martello S., Vigo D. Recent Advances on Two-Dimensional Bin Packing Problems // Discrete Applied Mathematics. 2002. Vol. 123. P. 379–396.
5. Zhang Y., Wang X., Liu Z. A Novel Approach for Solving 3D Bin Packing Problem by Integrating a Generative Adversarial Network with a Genetic Algorithm // Scientific Reports. 2024. DOI: <https://doi.org/10.1038/s41598-024-56699-7>.
6. DrogoZZZZ. 3D Bin Packing [Электронный ресурс]: GitHub-репозиторий проекта. URL: https://github.com/DrogoZZZZ/3D_bin_packing (дата обращения: 26.05.2026).